Physicomimetics in 3D with real vehicles

How the work is organised · 18 October 2026

This is how the conversion of *Physicomimetics* to BPP is carried out: where things live in the repo, the steps every task goes through, how progress is tracked, and how the work continues from one chat to the next. `BOOK.md` says *what* to build and in what order; this file says *how*.

1. Where things live

Everything stays under `demo/WyomingWill/ArtificialPhysics/`. Existing files do not move, so nothing that points at them breaks.

Folder or file	Holds
`BOOK.md`, `BOOK.pdf`, `book/`	The plan, the chapter notes and (in `book/theses/`) the thesis digests. `BOOK.md` is also the progress tracker
`WORKFLOW.md`, `WORKFLOW.pdf`	This file
`HANDOFF.md`	How William likes to work, build and run steps, the current state and the next step. A new chat starts from the repo alone
`FIXES.md`	Fixes, as now
`FINDINGS.md`, `FINDINGS.pdf`	The running record of experiments: confirmations of the book, new results, errors and divergences
`lib/` (new)	Shared code: the random number generator, batch runner, momentum and energy checks, metrics, force-law table, sensing model, and the world generator (G21), which builds every world from the book's presets to the capstone harbour
`quadcopter/` (exists), then `ground/`, `boat/`, `sub/`, `fixedwing/`	One folder per kind of vehicle, all behind the same interface (G18 in `BOOK.md`)
`ch02/` … `ch19/`, `capstone/`	The ports, one folder per chapter. Each has a README with the book's numbers, ours, and the choices made where the sources are silent or disagree
`probes/`, `checks/` (new)	The measurement probes from the quadcopter work, and one check script per port
`tools/`	`md2pdf.py`, which typesets `BOOK.md` and `WORKFLOW.md` as PDFs

The existing top-level demos (`Formations-*.lua`, `Uniform_Coverage.lua`) stay where they are, so they remain easy to find in BPP's file dialog.

2. How each task is done

Every task goes through the same steps. Each step is one patch.

1. **Read the sources.** There are four (five for Ch. 8–9), and each can be right where the others are wrong:
   - the chapter note in `book/` (and the book itself for anything unclear);
   - the original `.nlogo` model in `netlogo/`, the reference implementation;
   - **the Android app's version**, which was mostly built from the `.nlogo` files. It sometimes improves on them, and it can also contain errors, including ones introduced in porting. Its known history: the Uniform Coverage port copied the one-sided neighbour check and the dead-end retry (both fixed in BPP, not yet in the app); Honeycomb needed many rounds of fixes, and its port notes (shuffling agents every tick, NetLogo's `atan` convention) apply to every NetLogo port; Chemical Plume Tracing came from Zarzhitsky's C/GTK tool, not NetLogo, with a changed plume generator;

- the matching thesis digest;
- for Ch. 8–9, Zarzhitsky's original CPT code (PlumeGen, TraceGen), which gives the plume and robot settings the book omits. Its map makers are not used; our own world generator (`BOOK.md` §6) builds every world and writes his map format only so his tools can check ours.

Where the sources disagree, the README records which was followed and why.

2. **Build the 2D particle replica**, with a headless check script that compares our numbers with the book's and prints pass or fail.
3. **Fly it on vehicles.**
4. **Take it to 3D**, where that makes sense.

Each patch updates the task's README, its status in `BOOK.md` (new → partly → done) and `BOOK.pdf`, `FIXES.md` if anything was fixed, `FINDINGS.md` and `FINDINGS.pdf` with whatever it confirmed, found or corrected, and `HANDOFF.md` at the end of a milestone. Every script states near its top what each quantity it shows or prints means (formula, units, sign).

3. Checks protect earlier work

As the check scripts accumulate, running them all is a regression suite: a change to the shared code cannot quietly break an earlier result. "Measure before claiming" becomes something a script enforces. Checks use the shared random number generator with fixed seeds, so their results repeat exactly.

4. Patches

As before: one patch per step, with the full git commands, checked to apply cleanly to the current `origin/master` (fetched first, since master moves). Each patch says whether a rebuild is needed (Lua-only patches need none; new C++ files need `qmake` re-run). Experiments that change behaviour are switchable, with the old behaviour at 0, so they are easy to revert.

5. Chats and milestones

Long chats get slow and lose detail, so there is one chat per milestone. A milestone is a few build steps ending in a summary, and each chat ends by updating `HANDOFF.md`. The milestones follow the build order in `BOOK.md`:

Milestone	Build steps	Content
M0	0	The folders above, the shared random number generator with a seed setting, the batch runner, momentum and energy checks, the world generator's core and first book presets, Ch. 2's three tests
M1	1	Ch. 13's four fixes on the drones, each switchable, measured
M2	2	The vehicle interface, ground robot and surface boat, first cross-vehicle benchmark
M3	3–5	Fidelity checks of the existing formations and coverage, Perfect Formation, velocity commands and kinetic theory
M4 onward	6–12	The rest of the build order, ending with the capstone

6. What stays out of the repo

The book PDF, the theses and Zarzhitsky's CPT code are not William's to publish, so they stay out of the repo; attach them to a chat that needs them. The Android app's code lives in its own project; attach `ap.zip` when a task needs it. Our chapter notes and the thesis digests are our own summaries and are in `book/`.